Lesson 6: Victory Condition

Unity

VIDEO GAME CREATION

This week: a real win condition. Your job is to track how many enemies are left and show "You Win!" the moment the last one is destroyed - completing your game's Level 1 MVP milestone.

Open your own [YourName]Game project from Lesson 5 - don't start a new one. Everything this week builds on the row of enemies you already have.

Tip: no new assets this week - "You Win!" is shown with each engine's own basic text tools, not an image.

Setup

1. Open your [YourName]Game project from Lesson 5.
2. Create a new empty GameObject named GameManager. Create a new C# script named GameManager, attach it to this GameObject.

Level 1 - Minimal (MVP)

The moment the last enemy is destroyed, "You Win!" appears on screen. This completes your game's Level 1 MVP milestone - a complete, playable core loop.

3. In GameManager.cs, add a public static int enemiesRemaining, a public static bool hasWon, and a static method RegisterKill() that decrements the count and sets hasWon at zero. Add an OnGUI() that draws "You Win!" when hasWon is true (code below).
4. In Bullet.cs's OnTriggerEnter2D, call GameManager.RegisterKill() right where you already destroy the enemy.
5. In GameManager.cs's Start() (or a field default), set enemiesRemaining to match your EnemySpawner's enemyCount.
6. Press Play and destroy every enemy to test - "You Win!" should appear the instant the last one is destroyed.

```
// GameManager.cs:
public static int enemiesRemaining = 5;
public static bool hasWon = false;

public static void RegisterKill() {
    enemiesRemaining--;
    if (enemiesRemaining <= 0) hasWon = true;
}
```

```
void OnGUI() {
    if (hasWon) GUI.Label(new Rect(10, 10, 200, 30), "You Win!");
}

// Bullet.cs - add inside OnTriggerEnter2D, next to
Destroy(other.gameObject):
GameManager.RegisterKill();
```

Alternate Path: Visual Scripting

The same kill-tracking pattern works in a Script Graph instead of C#.

7. On the GameManager GameObject, add a Script Machine with a Scene Variable enemiesRemaining set to 5.
8. On the Bullet prefab's graph, right after the existing Destroy nodes in its trigger event, add a "Decrement" node on the scene's enemiesRemaining variable, followed by a Branch checking if it's <= 0.
9. On True, use an "On GUI" event node on GameManager's graph with a GUI Label node showing "You Win!", gated by a Scene Variable hasWon that the Branch sets to true.

Level 2/3 already work the same way as the C# path, since the counting logic lives in one place (the scene variable) either way.

Level 2 - Complete (Alpha/Beta)

The win text appears exactly once, exactly when the last enemy is destroyed - not early, not late, and not repeatedly.

- Destroy all 5 enemies and confirm "You Win!" appears exactly once, exactly when the last one is destroyed - not one enemy early, and not flickering or repeating.
- This already works correctly because RegisterKill() runs exactly once per actual hit (right next to the existing Destroy(other.gameObject) call), and hasWon only ever flips from false to true, never back.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: add an AudioClip field to GameManager, assign the provided sfx_win.wav to it in the Inspector, and call AudioSource.PlayClipAtPoint() the moment hasWon first becomes true inside RegisterKill().
- Visual: style OnGUI()'s label with a custom GUIStyle (bigger font size, centered) instead of the plain default look.
- Structure: keep all win-tracking (enemiesRemaining, hasWon, RegisterKill()) inside GameManager.cs - Bullet.cs only ever calls RegisterKill(), it never touches the counting logic directly.

If Something's Wrong

- "You Win!" never appears: confirm RegisterKill() is actually being called from Bullet.cs, and that the GameManager GameObject is present and active in the scene.
- "You Win!" appears too early: confirm enemiesRemaining's starting value actually matches your EnemySpawner's enemyCount.

- Nothing draws at all: OnGUI() only runs on a MonoBehaviour that's active in the scene - confirm GameManager.cs is attached to a GameObject that hasn't been disabled or deleted.

Want More?

Student Resource List > Unity section, or learn.unity.com